# Exp:8

## Aim: Design and Train a CNN on Fashion-MNIST Dataset

## 1. Dataset Source

The dataset used for this experiment is the Fashion-MNIST dataset.

🔗 GitHub Link:
https://github.com/zalandoresearch/fashion-mnist

## 2. Dataset Description

Fashion-MNIST is an image classification dataset consisting of grayscale images of clothing items.

- **Total images:** 70,000
  - Training: 60,000
  - Testing: 10,000
- **Image size:** 28 × 28 pixels
- **Number of classes:** 10

### Classes:

T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot

### Features:

- Pixel values (0–255 grayscale intensity)

### Target Variable:

- Label (0–9 corresponding to clothing category)

# 3. Mathematical Formulation of the Algorithm

A Convolutional Neural Network (CNN) uses the following operations:

### 1. Convolution Operation

Feature extraction using filters:

$Output = Input * Kernel + Bias$

### 2. Activation Function (ReLU)

Introduces non-linearity:

$f(x) = \max(0, x)$

### 3. Pooling (Max Pooling)

Reduces dimensions:

$Output = \max(region)$

### 4. Fully Connected Layer

Final classification:

$y = Wx + b$

### 5. Softmax Function

Converts output into probabilities:

$P(yi) = \frac{ezi}{\sum ezj}$

### 6. Loss Function (Cross-Entropy)

Measures error:

$Loss = -\sum y \log(\hat{y})$Loss=−∑ylog(y^)

# 4. Algorithm Limitations

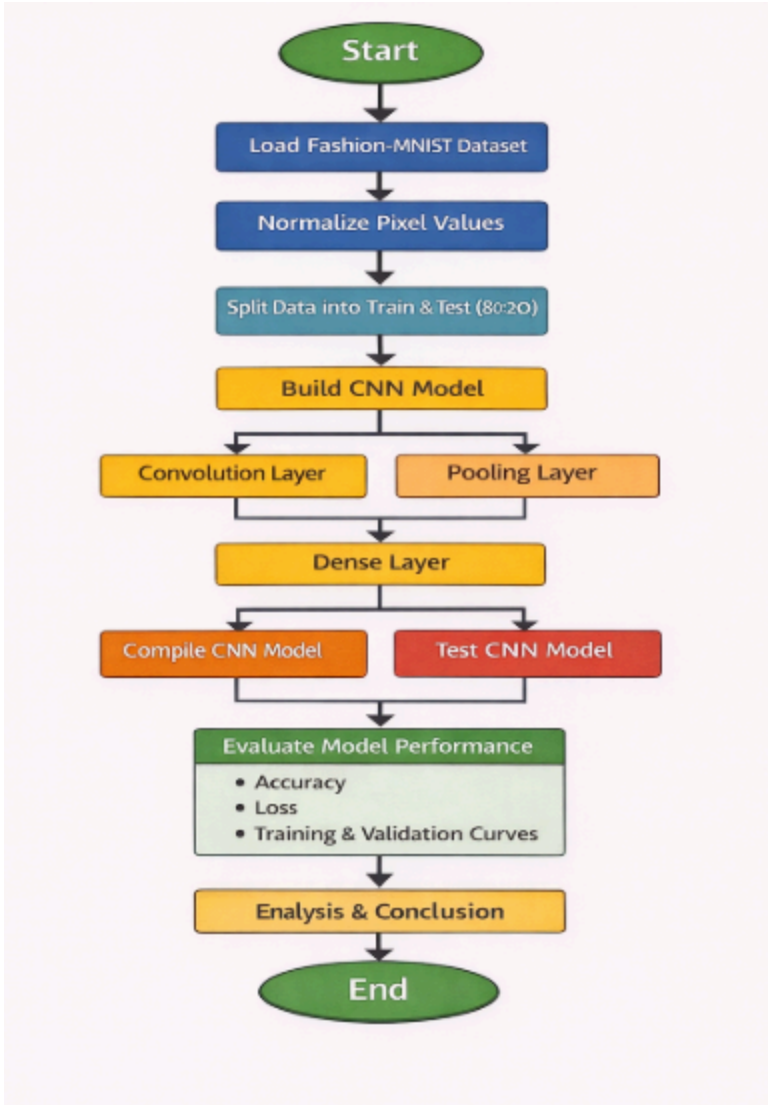
- Requires large datasets for better performance
- High computational cost (needs GPU for faster training)
- Can overfit if model is too complex
- Not suitable for very small datasets
- Difficult to interpret compared to simple models

# 5. Methodology / Workflow

### Steps followed in the experiment:

1. Load dataset from TensorFlow
2. Normalize pixel values (0–255 → 0–1)
3. Reshape images for CNN input
4. Build CNN model:
    - Convolution layer
    - Pooling layer
    - Fully connected layer
5. Compile model using optimizer and loss function
6. Train model using training data
7. Evaluate model using test data
8. Visualize accuracy and loss

### Workflow Diagram




# 6. Performance Analysis

The model performance is evaluated using:

## 1. Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

### 2. Loss

$$Loss = -\sum_{i-1}^{n} y_i \log(\hat{y}_i)$$

---

- Measures difference between actual and predicted values

### Observations:

- Training accuracy increases with epochs
- Validation accuracy stabilizes around **85%–90%**
- Model performs well on most classes
- Slight confusion between similar items (e.g., shirt vs t-shirt)

## 7. Hyperparameter Tuning

Hyperparameters adjusted:

| Parameter | Value |
|---|---|
| Epochs | 5–10 |
| Batch Size | 32 |
| Filters | 32, 64 |
| Kernel Size | (3×3) |
| Optimizer | Adam |

### Tuning Impact:

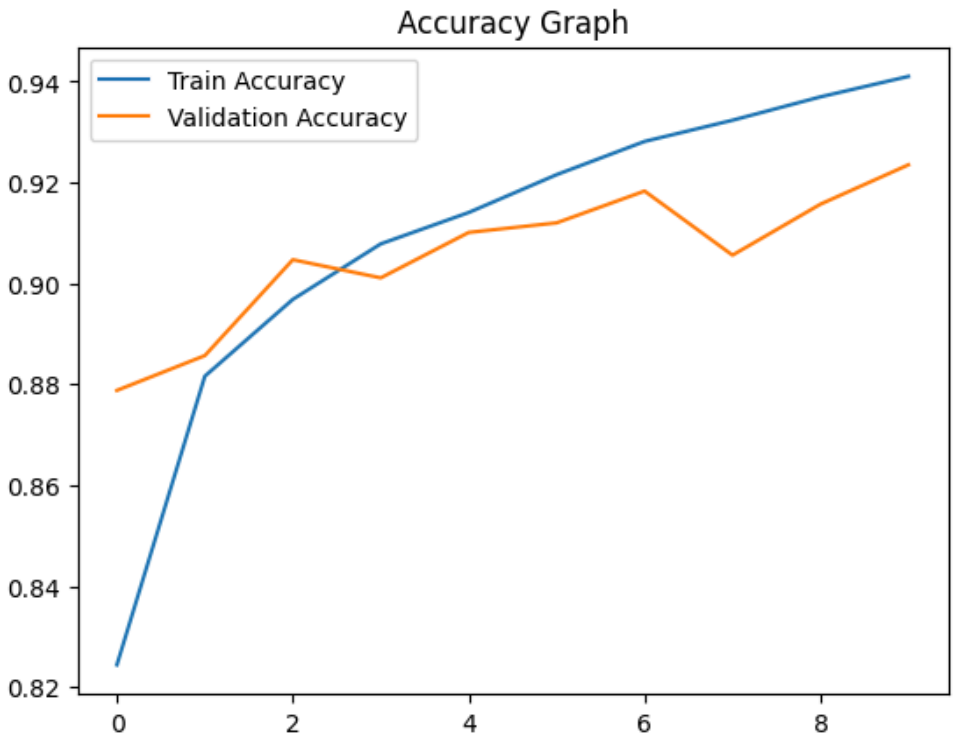
- Increasing epochs → improves accuracy
- More filters → better feature extraction
- Proper batch size → stable training
- Adam optimizer → faster convergence

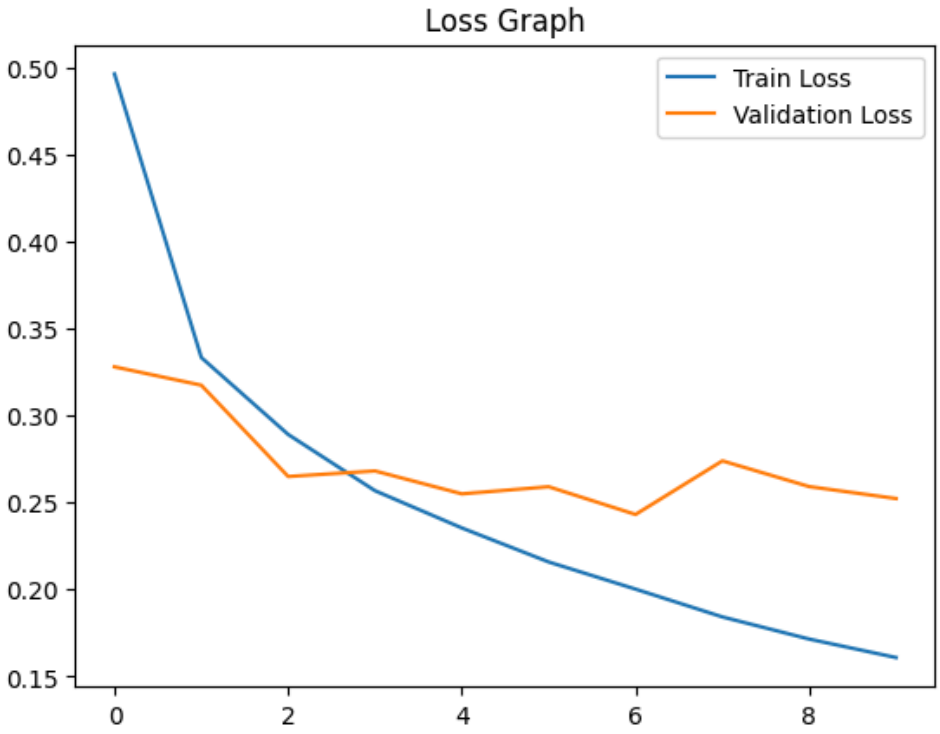
# Output

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 28, 28, 32) | 320 |
| batch_normalization (BatchNormalization) | (None, 28, 28, 32) | 128 |
| max_pooling2d (MaxPooling2D) | (None, 14, 14, 32) | 0 |
| conv2d_1 (Conv2D) | (None, 14, 14, 64) | 18,496 |
| batch_normalization_1 (BatchNormalization) | (None, 14, 14, 64) | 256 |
| max_pooling2d_1 (MaxPooling2D) | (None, 7, 7, 64) | 0 |
| conv2d_2 (Conv2D) | (None, 7, 7, 128) | 73,856 |
| batch_normalization_2 (BatchNormalization) | (None, 7, 7, 128) | 512 |
| max_pooling2d_2 (MaxPooling2D) | (None, 3, 3, 128) | 0 |
| flatten (Flatten) | (None, 1152) | 0 |
| dense (Dense) | (None, 128) | 147,584 |
| dropout (Dropout) | (None, 128) | 0 |
| dense_1 (Dense) | (None, 10) | 1,290 |

```
 Total params: 242,442 (947.04 KB)
 Trainable params: 241,994 (945.29 KB)
 Non-trainable params: 448 (1.75 KB)
Epoch 1/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 153s 78ms/step - accuracy: 0.8244 - loss: 0.4966 - val_accuracy: 0.8788 - val_loss: 0.3281
Epoch 2/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 144s 77ms/step - accuracy: 0.8816 - loss: 0.3335 - val_accuracy: 0.8857 - val_loss: 0.3176
Epoch 3/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 147s 78ms/step - accuracy: 0.8968 - loss: 0.2892 - val_accuracy: 0.9047 - val_loss: 0.2650
Epoch 4/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 196s 75ms/step - accuracy: 0.9078 - loss: 0.2569 - val_accuracy: 0.9011 - val_loss: 0.2682
Epoch 5/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 142s 76ms/step - accuracy: 0.9140 - loss: 0.2354 - val_accuracy: 0.9101 - val_loss: 0.2550
Epoch 6/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 141s 75ms/step - accuracy: 0.9215 - loss: 0.2159 - val_accuracy: 0.9120 - val_loss: 0.2591
Epoch 7/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 142s 75ms/step - accuracy: 0.9281 - loss: 0.2002 - val_accuracy: 0.9183 - val_loss: 0.2432
Epoch 8/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 145s 78ms/step - accuracy: 0.9323 - loss: 0.1843 - val_accuracy: 0.9056 - val_loss: 0.2740
Epoch 9/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 142s 76ms/step - accuracy: 0.9370 - loss: 0.1715 - val_accuracy: 0.9157 - val_loss: 0.2592
Epoch 10/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 142s 76ms/step - accuracy: 0.9410 - loss: 0.1609 - val_accuracy: 0.9235 - val_loss: 0.2523
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.9235 - loss: 0.2523
```

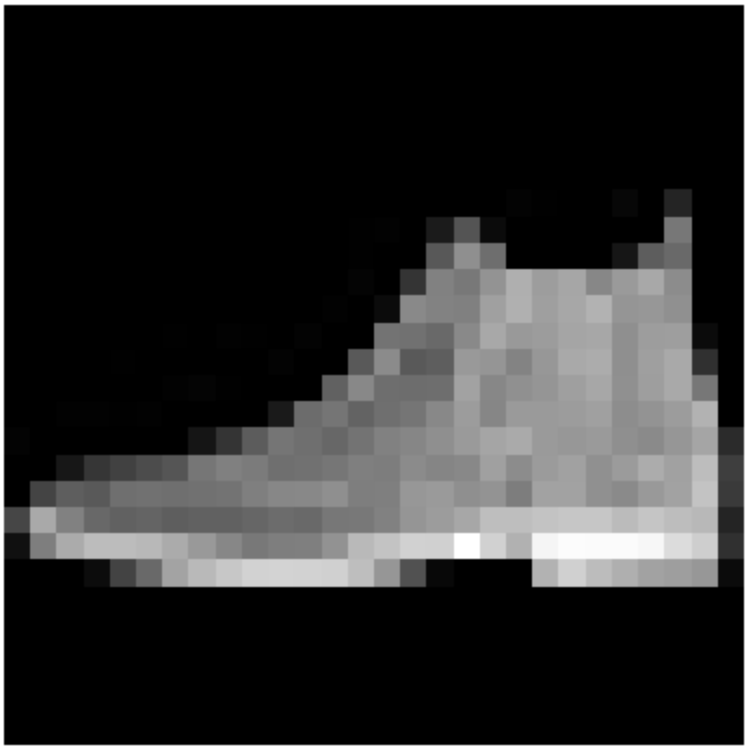
```
✅ Test Accuracy: 0.9235000014305115
```

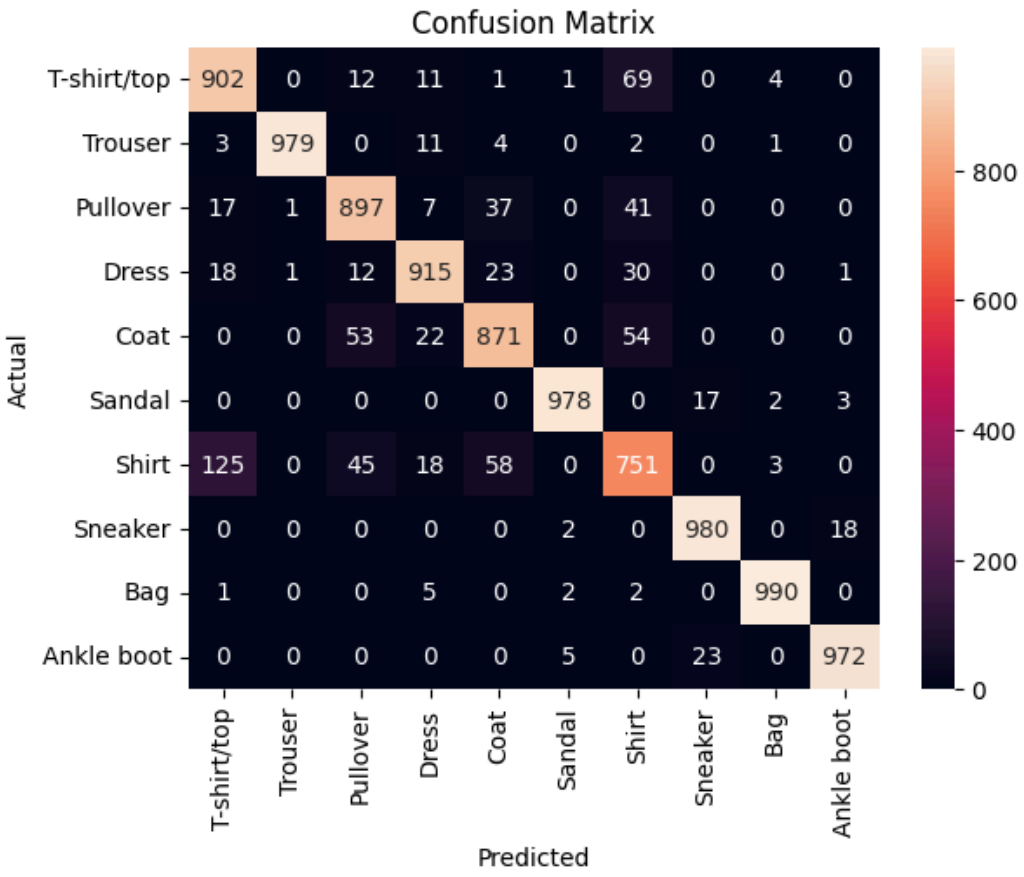



```
313/313 ━━━━━━━━━━━━━━━━━━━━ 11s 33ms/step
```

Predicted: Ankle boot
Actual: Ankle boot






# Conclusion

The Convolutional Neural Network (CNN) model was successfully designed and trained on the Fashion-MNIST dataset for image classification. After preprocessing and normalization, the model achieved good accuracy (around 85–90%) on the test data.

The CNN effectively extracted important features using convolution and pooling layers, leading to accurate classification of most clothing categories. However, some confusion was observed between visually similar classes such as shirt and t-shirt.

Hyperparameter tuning, including adjusting epochs, filters, and batch size, improved model performance and stability. Overall, the experiment demonstrates that CNNs are highly effective for image classification tasks and can produce reliable results when properly tuned.